The actor's business logic is minimal. A funds reservation is accepted if the balance is sufficient or denied otherwise, persisting `FundsReserved` or `FundsReservationDenied`, respectively. Adding funds is always accepted, and the `FundsAdded` event is persisted.

Listing 12.3 Command handler

```
object Wallet {
  ...

  def handleCommands(
      state: State,
      command: Command): ReplyEffect[Event, State] = {
    command match {
      case ReserveFunds(amount, replyTo) =>
        if (amount <= state.balance)                          ❶
          Effect
            .persist(FundsReserved(amount))
            .thenReply(replyTo)(state => Accepted)
        else
          Effect
            .persist(FundsReservationDenied(amount))
            .thenReply(replyTo)(state => Rejected)
      case AddFunds(amount, replyTo) =>
        Effect
          .persist(FundsAdded(amount))
          .thenReply(replyTo)(state => Accepted)
      case CheckFunds(replyTo) =>
        Effect.reply(replyTo)(CurrentBalance(state.balance))  ❷
    }
  }
  ...
}
```

❶ Reserves funds if the amount is less than the balance

❷ Checking funds has nothing to persist.

Finally, the event handler updates the balance when you add or reserve funds; otherwise the state remains unchanged:

```
object Wallet {
  ...
```